

Assignment 2 Part 3 - Task 3

paradox.c output

```
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ gcc -O2 paradox.c -lpthread -o paradox
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ ./paradox 1

Threads: 1
Iterations: 100000000

A threads= 1 time= 0.6578 forward=1e+19 reverse=1e+19 difference=0
B threads= 1 time= 0.6352 forward=1e+19 reverse=1e+19 difference=0
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ ./paradox 2

Threads: 2
Iterations: 100000000

A threads= 2 time= 0.8408 forward=0 reverse=0 difference=0
B threads= 2 time= 0.6988 forward=0 reverse=0 difference=0
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ ./paradox 4

Threads: 4
Iterations: 100000000

A threads= 4 time= 0.8904 forward=199.9999967908184 reverse=0 difference=200
B threads= 4 time= 0.6227 forward=199.9999967908184 reverse=0 difference=200
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ ./paradox 8

Threads: 8
Iterations: 100000000

A threads= 8 time= 1.4740 forward=199.9999967908184 reverse=0 difference=200
B threads= 8 time= 0.7407 forward=199.9999967908184 reverse=0 difference=200
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$ ./paradox 12

Threads: 12
Iterations: 100000000

A threads=12 time= 1.1258 forward=199.9999967908184 reverse=0 difference=200
B threads=12 time= 0.6756 forward=199.9999967908184 reverse=0 difference=200
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC_410/A2_3$
```

Expectations and observations

I thought more threads would make it faster and that forward and reverse would be the same. B stayed about the same time and A got slower with more threads. With 4 or more threads, forward was about 200 and reverse was 0.

What explains it

Each thread runs the whole loop, so more threads doesn't make it faster. In A the sums are stored right next to each other, so the threads get in each other's way. B spreads them out, so it's faster. The sums don't match because the small numbers get lost when they're added to a huge number, so the order changes the answer.

How to improve it

Use a local variable inside the loop instead of writing to the shared sum every time. Each thread adds to its own local variable and only saves it to the sum once at the end. That way the threads aren't constantly writing to memory next to each other, so A should run about as fast as B.